

Networking, HTTP and REST

SWEN1 | BIF3

Table of Contents

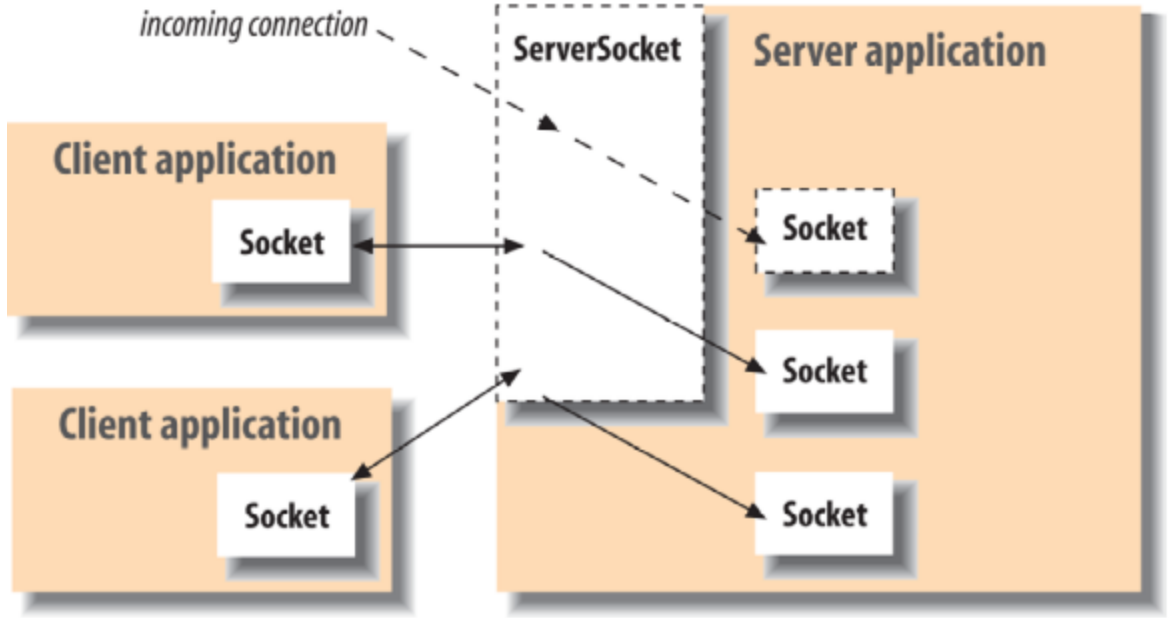
- Network Communication
- HTTP
- REST



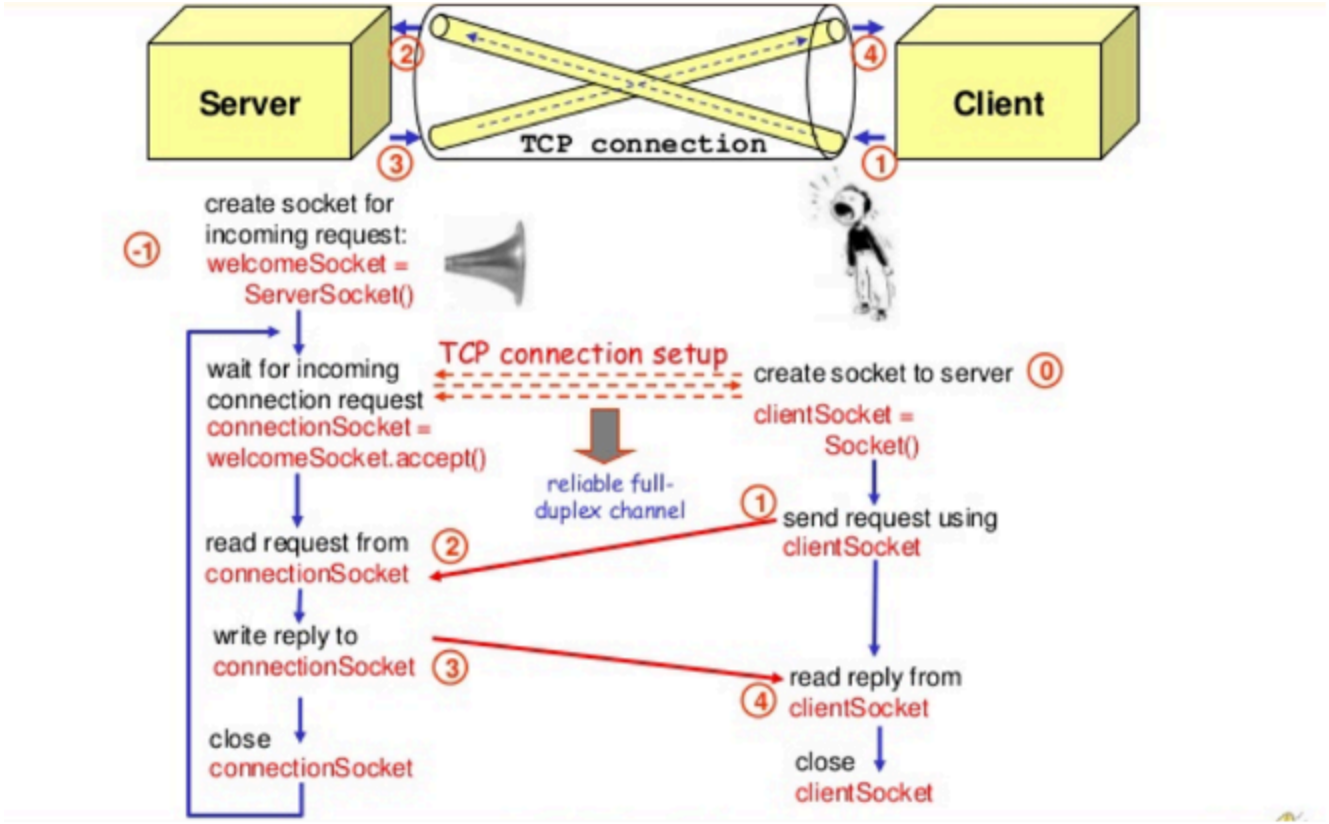
Network Communication

Sockets

- A socket is a 2 way communication channel between 2 applications across the network.
- It is bound to a port which is the registered to this single application.



Socket Communication



Server Sockets in C#

- The Socket class handles the (TCP) Communication in .NET
- The TcpListener / TcpClient class provides a TCP specific implementation with less protocol options and hence less code for TCP based communication

Creating a socket instance:

```
serverSock = new Socket (
  AddressFamily.InterNetwork, // IPv4
  SocketType.Stream,          // Stream sends byte array based Data
                              // and supports reliable way communication
  ProtocolType.Tcp            // TCP
}
```

Bind the socket to an IP Address and Port:

```
// loopback (127.0.0.1), Port: 10101
serverSock.Bind(new IPEndPoint(IPAddress.Loopback, 10101)) ;
```

Server Sockets in C#

- To enable the server to accept client connections it is necessary to call the Listen and Accept method
- The Accept method returns a new Socket object which is used to handle the communication with the client (send, receive)

```
serverSock.Listen(5);
clientSock = serverSock.Accept();
```

Sending Data via Send method by providing a byte[]

```
clientSock.Send(Encoding.ASCII.GetBytes("Hello World"));
```

Receiving Data via Receive method by retrieving a byte[]

```
int length = clientSock.Receive(bufferAsByteArray);
string input = Encoding.ASCII.GetString(bufferAsByteArray, 0, length);
```

Client connection via Sockets

As a simple TCP client:

```
public static void Read()
{
  var s = new TcpClient();
  s.Connect("technikum-wien.at", 80);
  var stream = s.GetStream();
  var sr = new StreamReader(stream);
  while (!sr.EndOfStream)
  {
    var line = sr.ReadLine);
    Console.WriteLine(line);
  }
}
```

Server Sockets in Java

```
try (ServerSocket listener = new ServerSocket (23)) {
  while (true) {
    Socket serviceSocket = listener.accept();
    new Thread(() -> {
      try (BufferedReader input = new BufferedReader (new InputStreamReader (serviceSocket.getInputStream()));
           PrintStream output = new PrintStream(serviceSocket.getOutputStream())) {
        while (true) {
          String line = input.readLine();
          if (line == null || line.equalsIgnoreCase("quit"))
            break;
          System.out.println("EchoServer: echo " + line);
          output.println(line);
        }
      } catch (I0Exception ex) {
        System.err.println("EchoService: ERROR " + ex) ;
      }
    }).start();
  }
} catch (IOException e) {
  System.err.println("EchoServer: ERROR " + e);
}
```

Client Sockets in Java

```
Socket s = new Socket();
s.connect(new InetSocketAddress("www.technikum-wien.at", 80));

BufferedWriter out = new BufferedWriter(new OutputStreamWriter(s.getOutputStream()));
out.write("GET / HTTP/1.1\r\n");
out.write("\r\n");
out.flush();

BufferedReader in = new BufferedReader(new InputStreamReader(s.get InputStream()));
String line = null;
while ((line = in.readLine()) != null) {
  System.out.println(line);
}
```



HTTP

Hyper-Text Transfer Protocol

- The HTTP protocol (hyper text transfer protocol) was designed to transport data over the network. It is especially used in the World Wide Web (WWW) used by browsers.
- It is an application layer protocol based on TCP/IP designed to be stateless implementing a request/response pattern. Hereby a client (browser) makes a request to a web-server who answers with a response.

History:

- 1989 Tim Berners-Lee at CERN
- was created together with URL and HTML
- wanted to “link” research papers with references

HTTP Overview

- Request sends HTTP headers to a web-server (in the example starting with >)
- Request transmission ends with a newline
- Response has two phases
  - Content Header (meta-data about the body of the message like encoding, mime-type)
  - Content Body (can be any kind of file content, but often html, CSS, JavaScript)
- no implicit session logic (implementation of a session-id in code possible; application layer)
- Extensible because plain text format
  - Request method (here in the sample GET)
  - Header information
  - status codes (see HTTP-StatusCodes)
  - data content

HTTP Request

C:\>curl -v localhost:8080

```
* Rebuilt URL to: localhost:8080/
*   Trying ::1...
* TCP_NODELAY set
* Connected to localhost (::1) port 8080 (#0)
> GET / HTTP/1.1
> Host: localhost:8080
> User-Agent: curl/7.55.1
> Accept: */*
>
```

HTTP Response

```
< HTTP/1.1 200 OK
< X-Powered-By: Express
< Accept-Ranges: bytes
< Cache-Control: public, max-age=0
< Last-Modified: Thu, 30 Jul 2020 08:27:36 GMT
< ETag: W/"3b7f-1739ed57861"
< Content-Type: text/html; charset=UTF-8
< Content-Length: 15231
< Date: Thu, 30 Jul 2020 08:30:03 GMT
< Connection: keep-alive
<
<!doctype html>
<html lang="en">
<head><title>test page</title></head>
<body>test</body>
</html>
```

What happens in the network when...

...we enter a URL in the browser:

- DNS resolves us the domain name to an ip
- the port is determined (standard http: 80; https: 443)
- TCP Socket opened to the server
- GET request is sent
- Response received (static or dynamically generated content)
- Socket closed
- in case of many resources (like 100 pictures on a page) for each resource a socket will be opened

Network related improvements of HTTP

First improvements in HTTP 1.1:

- Keep-Alive Flag so that connection is kept open
- HTTP-Pipelining to reuse already open sockets

Some issues of the original HTTP Protocol are fixed in HTTP 2.

- reuse of sockets (so only 1 socket is open even for 100 images on a page)
- reduced header information (e.g. UserAgent is transmitted only once, because we assume it won't change during an open socket)
- automatic data compression
- HTTP2 server push (the server can send resources before they are requested)

Network related improvements in HTTP 3

HTTP/3 (based on a previous RFC draft), then named “Hypertext Transfer Protocol (HTTP) over QUIC”. (see zdnet)

QUIC (pronounced “quick”) is a general-purpose transport layer network protocol initially designed by Jim Roskind at Google, implemented, and deployed in 2012, announced publicly in 2013 as experimentation broadened, and described to the IETF.
QUIC is used by more than half of all connections from the Chrome web browser to Google’s servers. QUIC is already supported by all major browsers. It is faster because it establishes a number of multiplexed connections between two endpoints over User Datagram Protocol (UDP). (Wikipedia)

Cookies

- To store information at the client (e.g.: a session-id). Cookie information is embedded in the header. Can be deleted by the user.
- Currently the HTML5 local store is more common to store data. Cookies are still a good solution in terms of security because they can be flagged as “http-only” and “secure-only”

Typical HTTP-Methods

- GET: requests a resource
- HEAD: requests the header information of a resource
- POST: can also send data to the server (data in http content of a request)
- PUT: replaces a representation of the target resource with the request payload.
- DELETE: removes a specified resource
- OPTIONS: describe the communication options for the target resource

Keep in mind:

- don’t misuse the methods (design smell)
- e.g.: GET with request parameters to write data
  - https://www.google.at/?hl=de



REST

Guiding Principles of REST

- Client–server
- Stateless
- Cacheable
- Uniform interface
- Layered system
- Code on demand (optional)

Advantages:

- Loose coupling between frontend and backend
- As long as the API stays the same, technologies can be changed

Resource

- The key abstraction of information in REST is a resource
- Any information that can be named can be a resource
  - document or image
  - temporal service
  - collection of other resources
  - non-virtual object (e.g. a student)
- The state of the resource at any particular timestamp is known as resource representation

Resource (HTTP) methods and CRUD

GET (=Read)

- Get a representation of the target resource's state

POST (=Create)

- Let the target resource process the representation enclosed in the request

PUT (=Update)

- Create or replace the state of the target resource with the state defined by the representation enclosed in the request

DELETE (=Delete)

- Delete the target resource's state

REST example

Get book resource 11 - 30

Method	URL	Description
GET	/books	Get all book resources
GET	/books/123	Get the book resource of book id 123
POST	/books	Create a new book resource
PUT	/books/123	Update the book resource of book id 123
DELETE	/books/123	Delete the book resource of book id 123
GET	/authors/321/books	Get all book resources of author resource id 321

REST example sorting, filtering, pagination

Get book resource 11 - 30

Method	URL	Description
GET	/books?sort=+title	Get all book resources sorted by title ascending
GET	/books?title=1984	Get all book resources filtered by title „1984“
GET	/books?page=4	Get all book resources of page 4
GET	/books?offset=10&max=20	Get book resource 11 - 30